



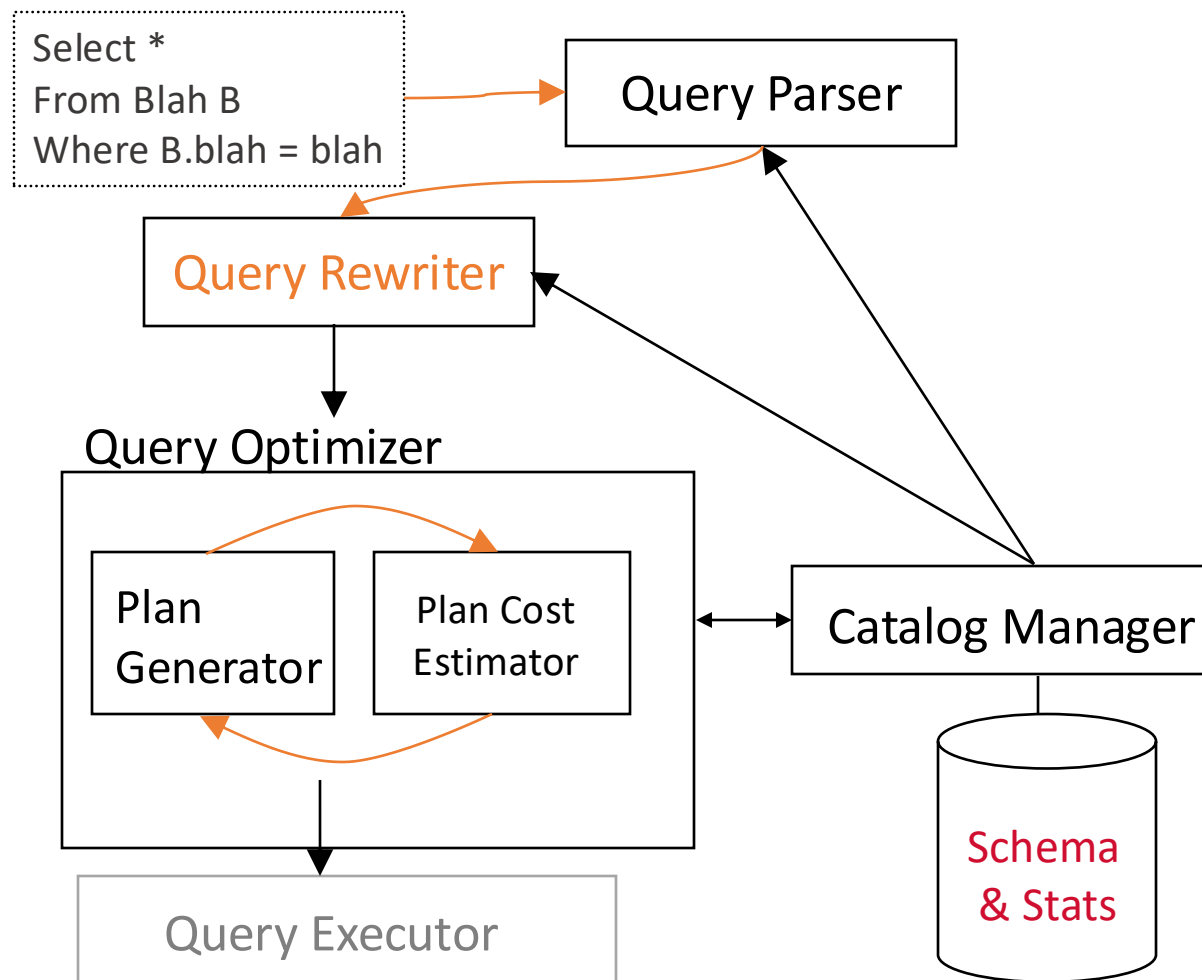
Introduction to Database Systems

2024-Fall



4. Database Management Systems (3/5)

4.3 Query Parsing & Optimization

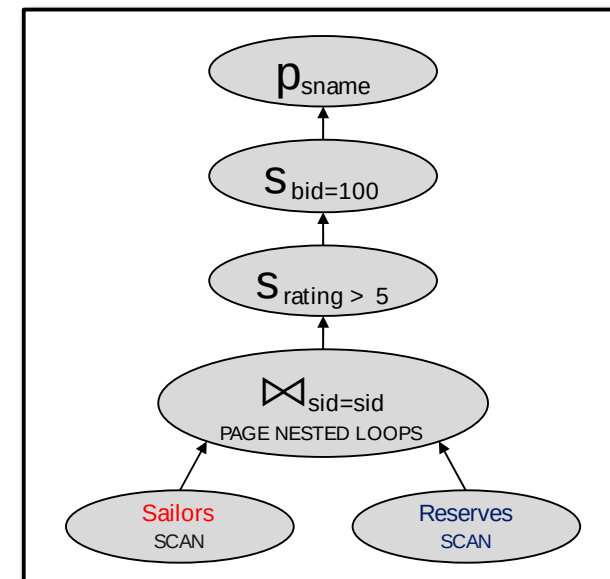


- **Query parser**
 - Checks correctness, authorization
 - Generates a parse tree
 - Straightforward
- **Query rewriter**
 - Converts queries to canonical form
 - flatten views
 - subqueries into fewer query blocks
 - Weak spot in many open-source DBMSs
- **“Cost-based” Query Optimizer**
 - Optimizes 1 query block at a time
 - Select, Project, Join
 - GroupBy/Agg
 - Order By (if top-most block)
 - Uses catalog stats to find least-“cost” plan per query block

4.3 Query Optimization Overview

- Query block can be converted to relational algebra
- Relational Algebra converts to tree
- Each operator has implementation choices
- Operators can also be applied in different orders!

```
SELECT S.sname  
  FROM Reserves R, Sailors S  
 WHERE R.sid=S.sid  
       AND R.bid=100  
       AND S.rating>5
```



$$\pi_{(sname)} \sigma_{(bid=100 \wedge rating > 5)} (Reserves \bowtie Sailors)$$



4.3 Query Optimization: The Goal

- Optimization goal:
 - Ideally: Find the plan with least actual cost.
 - Reality: Find the plan with least estimated cost.
 - And try to avoid really bad actual plans!

查询优化是**相对而言**的，可能的执行策略很多，穷尽代价很大，不能片面追求绝对的最优。



4.3 The processing of database query languages

- Interpreted execution (解释执行)
- Compiled execution (编译执行)
- In what situations are interpreted and compiled methods applicable?
 - For common routine transactions, the compiled method can enhance performance.
 - For brief ad-hoc queries, the interpreted method is flexible and practical.



4.3 Query Optimizations

- Algebra Optimization
- Physical Optimization
- Cost-estimation*



4.3.1. Algebra Optimization

- Algebra optimization performs *equivalent transformations*(等效变换) on queries to reduce execution costs.
- The principle of algebraic optimization is to minimize the size of *intermediate results* during the query process.
- Selection and projection operations can usually effectively reduce the size of relations.
- Joins, Cartesian products, and union operations are likely to generate large intermediate query results.
- Therefore, *selections and projections should be performed first; joins between smaller relations should be done before those between larger relations*; and it's even necessary to *identify common expressions* in queries to avoid redundant computations.



Relational Algebra Equivalences

- ***Selections:***

- $\sigma_{c1 \wedge \dots \wedge cn}(R) \equiv \sigma_{c1}(\dots(\sigma_{cn}(R))\dots)$ (cascade)
- $\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$ (commute)

- ***Projections:***

- $\pi_{a1}(R) \equiv \pi_{a1}(\dots(\pi_{a1, \dots, an-1}(R))\dots)$ (cascade)

- ***Cartesian Product***

- $R \times (S \times T) \equiv (R \times S) \times T$ (associative)
- $R \times S \equiv S \times R$ (commutative)



Are Joins Associative and Commutative?

- Consider $R(a,z)$, $S(a,b)$, $T(b,y)$
 - $(S \bowtie_{b=b} T) \bowtie_{a=a} R \not\equiv S \bowtie_{b=b} (T \bowtie_{a=a} R)$ (*not legal!!*)
 - $(S \bowtie_{b=b} T) \bowtie_{a=a} R \not\equiv S \bowtie_{b=b} (T \times R)$ (*not the same!!*)
 - $(S \bowtie_{b=b} T) \bowtie_{a=a} R \equiv S \bowtie_{b=b \wedge a=a} (T \times R)$ (*the same!!*)



Equivalent transformations

$$1. \sigma_{c1 \text{ AND } c2 \dots \text{ AND } cn}(R) \equiv \sigma_{c1}(\sigma_{c2}(\dots(\sigma_{cn}(R))\dots))$$

$$2. \sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c2}(\sigma_{c1}(R))$$

$$3. \prod_{list_1}(\prod_{list_2}(\dots(\prod_{list_n}(R))\dots)) = \prod_{list_1}(R),$$

$$list_1 \subseteq list_2 \subseteq \dots \subseteq list_n$$

$$4. \prod_{A1, A2, \dots, An}(\sigma_c(R)) \equiv \sigma_c(\prod_{A1, A2, \dots, An}(R))$$

$$Attr(C) \subseteq \{A1, A2, \dots, An\}$$



Equivalent transformations

5. $\mathbf{R} \bowtie \mathbf{S} = \mathbf{S} \bowtie \mathbf{R}$

6. $\sigma_c(R \bowtie S) \equiv (\sigma_c(R)) \bowtie S,$

$$\text{Attr}(c) \subseteq \text{Attr}(R)$$

7. $\sigma_{c1 \text{ AND } c2}(R \bowtie S) \equiv (\sigma_{c1}(R)) \bowtie (\sigma_{c2}(S)),$

$$\text{Attr}(c1) \subseteq \text{Attr}(R), \text{Attr}(c2) \subseteq \text{Attr}(S)$$



Equivalent transformations

8. Suppose the attribute set: $L = \{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_n\}$

$$\{A_1, A_2, \dots, A_n\} \subseteq Attr(R)$$

$$\{B_1, B_2, \dots, B_n\} \subseteq Attr(S)$$

$$\pi_L(R \bowtie_C S) \equiv (\pi_{A_1, A_2, \dots, A_n}(R) \bowtie_C \pi_{B_1, B_2, \dots, B_n}(S))$$

$$9. \theta \in \{\cup, \cap, -\} \quad \sigma_c(R \theta S) \equiv (\sigma_c(R)) \theta (\sigma_c(S))$$

$$10. \theta \in \{\cup, \cap, -\} \quad \pi_L(R \theta S) \equiv (\pi_L(R)) \theta (\pi_L(S))$$

$$11. \theta \in \{\bowtie, \times, \cup, \cap\},$$

$$(R \theta S) \theta T \equiv R \theta (S \theta T)$$



Some Common Heuristics: Selections

- Selection cascade and pushdown
 - Apply selections as soon as you have the relevant columns
 - Example:
 - $\pi_{\text{sname}} (\sigma_{\text{bid}=100 \wedge \text{rating} > 5} (\text{Reserves} \bowtie_{\text{sid}=\text{sid}} \text{Sailors}))$
 - $\pi_{\text{sname}} (\sigma_{\text{bid}=100} (\text{Reserves}) \bowtie_{\text{sid}=\text{sid}} \sigma_{\text{rating} > 5} (\text{Sailors}))$



Schema for Examples

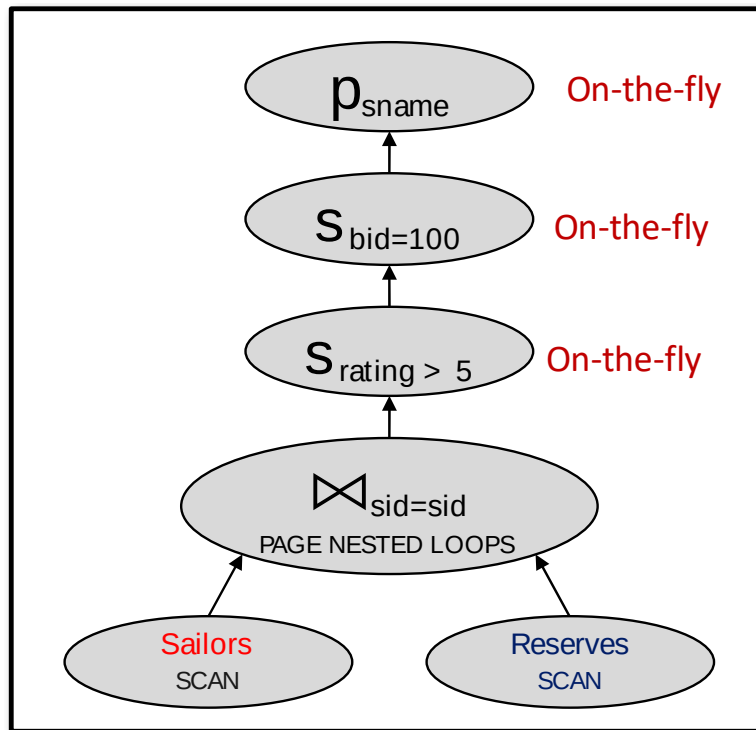
Sailors (*sid*: integer, *sname*: text, *rating*: integer, *age*: real)

Reserves (*sid*: integer, *bid*: integer, *day*: date, *rname*: text)

- Reserves:
 - Each tuple is 40 bytes long, 100 tuples per page, 1000 pages.
 - Assume there are 100 boats
- Sailors:
 - Each tuple is 50 bytes long, 80 tuples per page, 500 pages.
 - Assume there are 10 different ratings
- Assume we have 5 pages to use for joins.

Example 1

- Here's a reasonable query plan:



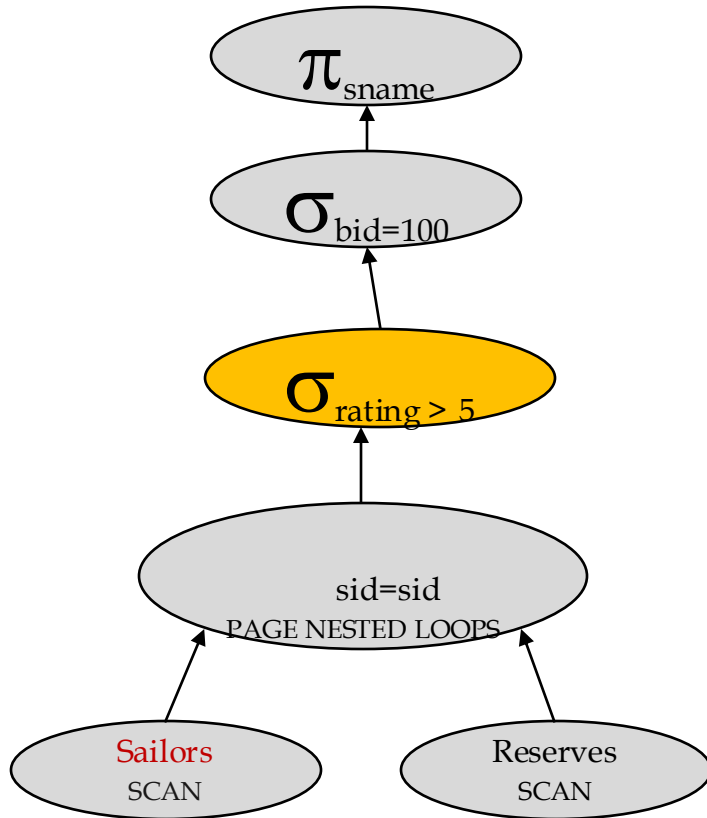
```

SELECT S.sname
FROM Reserves R, Sailors S
WHERE R.sid=S.sid
      AND R.bid=100
      AND S.rating>5
  
```

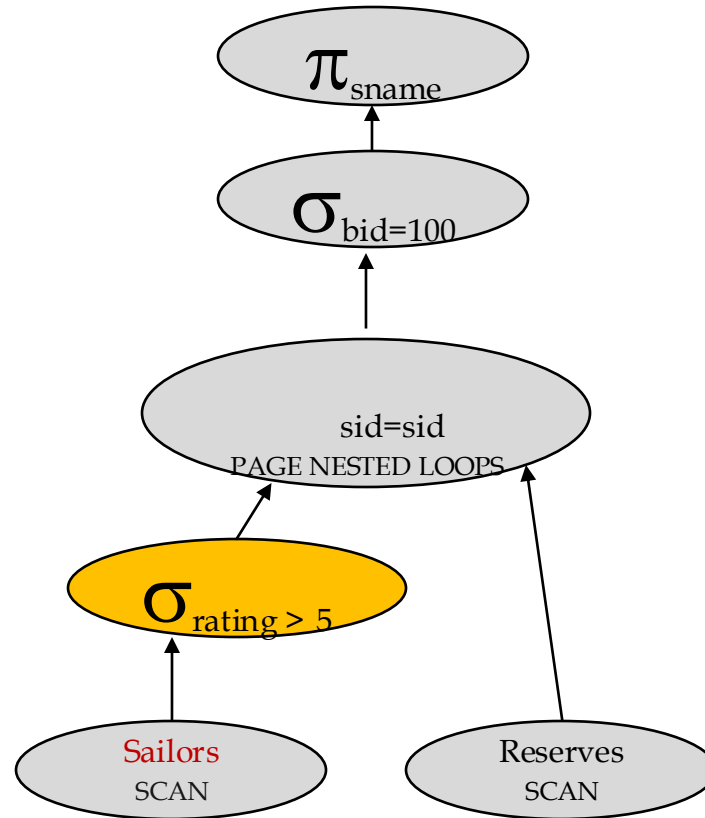
- Let's estimate the cost:
- Scan Sailors (500 IOs)
- For each page of Sailors, Scan Reserves (1000 IOs)
- Total: $500 + 500 * 1000$
 - 500,500 IOs**

Misses several opportunities: selections could be 'pushed' down, and no use made of indexes
Goal of optimization: Find faster plans that compute the same answer.

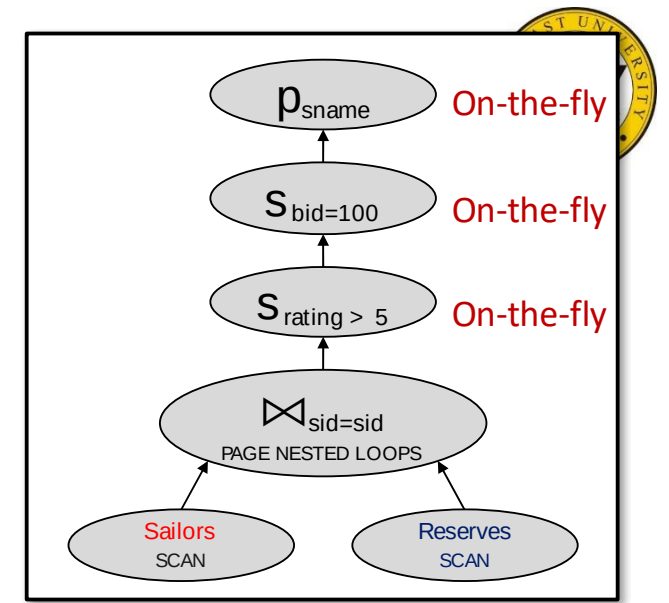
Plan1: Selection **Pushdown**



500,500 IOs



Cost?
250,500 IOs



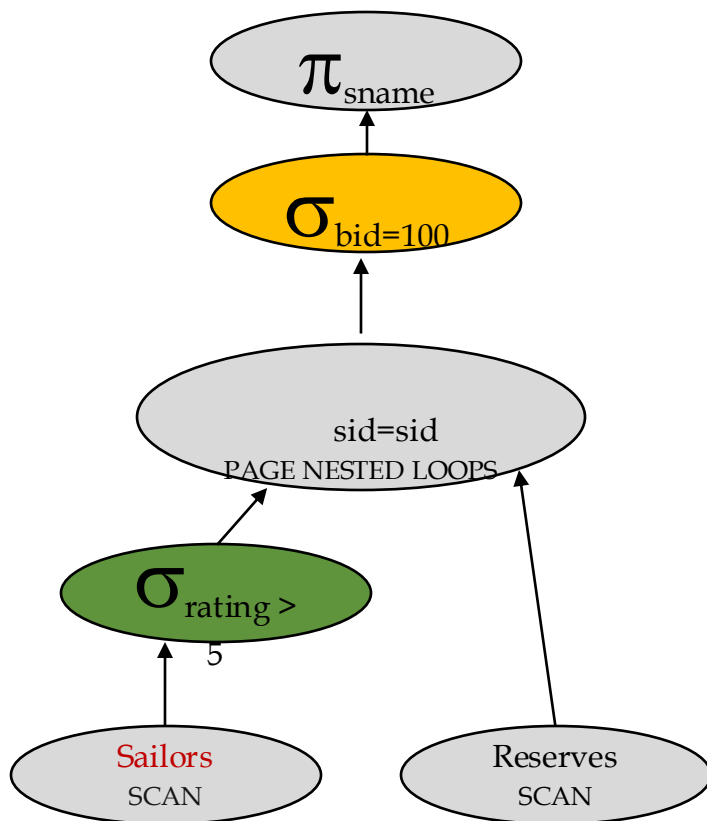
- Let's estimate the cost:
- Scan Sailors (500 IOs)
- For each page of high-rated Sailors, Scan Reserves (1000 IOs)

Total: 500 + ???*1000

Total: 500 + 250*1000

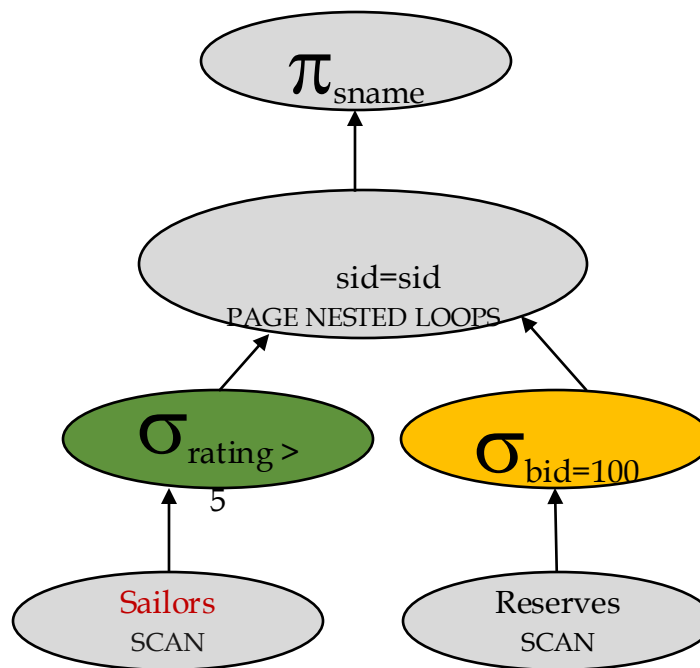


Plan2: More Selection Pushdown



250,500 IOs

500 + 250*1000



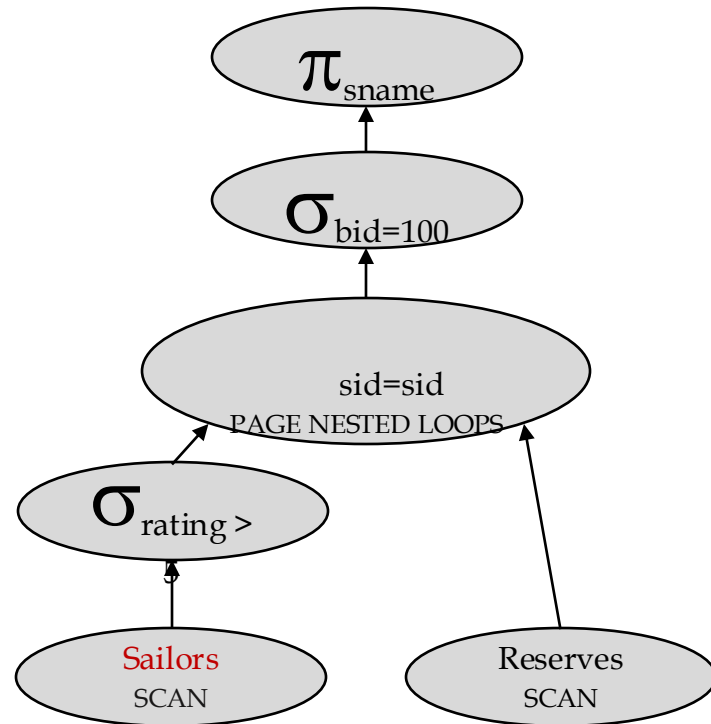
Cost???

Let's estimate the cost:

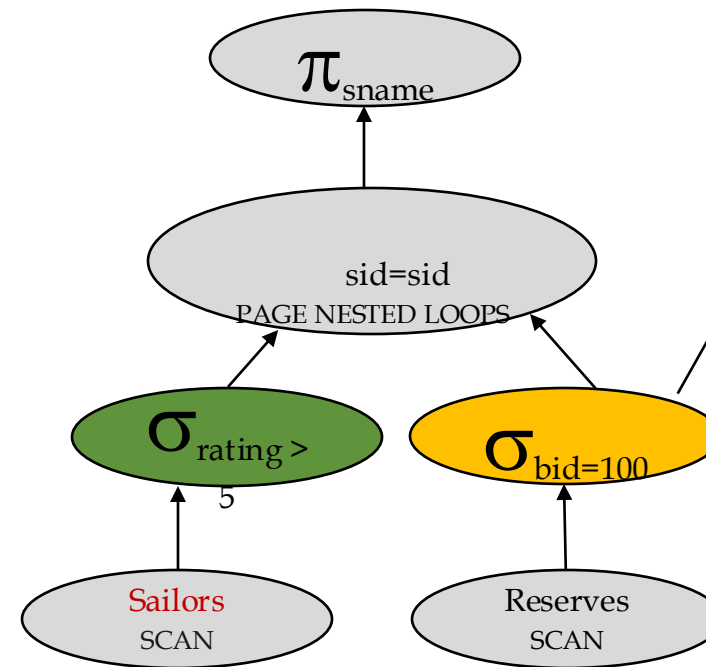
- Scan Sailors (500 IOs)
- For each pageful of high-rated Sailors,
Do what? (??? IOs)
- Total: 500 + 250*???
- Total: 500 + 250*1000!



More Selection Pushdown Analysis



250,500 IOs



250,500 IOs

Pushing a selection into the inner loop of a nested loop join doesn't save I/Os! Essentially equivalent to having the selection above.



Some Common Heuristics: Projections

- Projection cascade and pushdown
- Keep only the columns you need to evaluate downstream operators
- Example:
 - $\pi_{\text{sname}} \sigma_{(\text{bid}=100 \wedge \text{rating} > 5)} (\text{Reserves} \bowtie_{\text{sid}=\text{sid}} \text{Sailors})$
 - $\pi_{\text{sname}} (\pi_{\text{sid}} (\sigma_{\text{bid}=100} (\text{Reserves})) \bowtie_{\text{sid}=\text{sid}} \pi_{\text{sname}, \text{sid}} (\sigma_{\text{rating} > 5} (\text{Sailors})))$



Some Common Heuristics: Joins

- Join small relations first
 - Reduce the size of intermediate results
 - Save memory and I/O overhead
 - Improve cache efficiency
 - Speed up query execution



代数优化的基本步骤:

- 1.以SELECT子句对应投影操作，以FROM字句对应笛卡尔乘积，以WHERE子句对应选择操作，生成原始查询树。
- 2.应用变换规则2)、6)、7)、9)、10)，尽可能将选择条件移向树叶方向。
- 3.应用连接、笛卡尔乘积的结合律，按照小关系优先做的原则，重新安排连接（笛卡尔乘积）的顺序。
- 4.如果笛卡尔乘积后还须按连接条件进行选择操作可将两者组合成连接操作。
- 5.对每个叶节点加必要的投影操作，以消除对查询无用的属性。



When faced with nested queries

- 嵌入的查询块与上层查询无关
 - 从最低层查询开始，用上述步骤和规则，逐层计算各查询块所等效的关系，直至求出查询结果。
- 嵌入的查询块与上层查询有关
 - 一般用代入法。

例如：

```
SELECT A1
FROM R1
WHERE R1.A2 比较符 CONST1
      AND R1.A3 IN
      (SELECT A4
       FROM R2
       WHERE R2.A5 比较符 R1.A1)
```

逐个代入找出所有符合条件的A1





4.3.2. Optimization based on access path rules

- Selection
- Join
- Projection
- Set operation
- Combined operations



Schema for Examples

Sailors (*sid*: integer, *sname*: text, *rating*: integer, *age*: real)

Reserves (*sid*: integer, *bid*: integer, *day*: date, *rname*: text)

- Reserves:
 - Each tuple is 40 bytes long, 100 tuples per page, 1000 pages.
 - Assume there are 100 boats
- Sailors:
 - Each tuple is 50 bytes long, 80 tuples per page, 500 pages.
 - Assume there are 10 different ratings
- Assume we have 5 pages to use for joins.



Select

- The execution strategy of selection operations is related to the *selection condition*, *available access paths*, and *the proportion of selected tuples* in the entire relation.
- Conditions
 - Equal =
 - Range <>
 - Set IN, EXISTS, NOT EXISTS
 - Combined Condition AND, OR



Some Common Heuristics: Selection

实现方法：顺序扫描、尽量利用散列索引等方法。

- (1) 对于小关系，顺序扫描。
- (2) 若无索引、散列等存取路径可用，或估计选取的元组数占关系的比例较大（大于20%）且有关属性上无簇集索引，顺序扫描。
- (3) 对于主键的等值选择，优先选用主键的索引或散列。
- (4) 对于非主键的等值选择，若选取的元组数占关系的比例较小（小于20%），可以用无序索引；否则只能用簇集索引或顺序扫描。



Some Common Heuristics: Selection

(5) 对于范围条件，先通过索引找到范围的边界，再通过索引的顺序集沿相应方向搜索，如中选的元组数在关系中所占比例较大，宜采用簇索引或顺序扫描。

(6) 对于用AND连接的合取选择条件：

优先选用多属性索引

若有多个可用的次索引，可用预查找处理，最后做其余条件检查

个别条件可用 (3) (4) (5) 之一，求得相应组，再将这些元组用其它条件筛选

顺序扫描

(7) 用OR连接的析取选择条件，尚无好的方法。只能按其中各个条件分别选出一个元组集，再求这些元组集的并。

(8) 有些选择操作仅需访问索引就可得到结果，比如索引属性的最大值、最小值等，则应优先利用索引，而不访问数据。

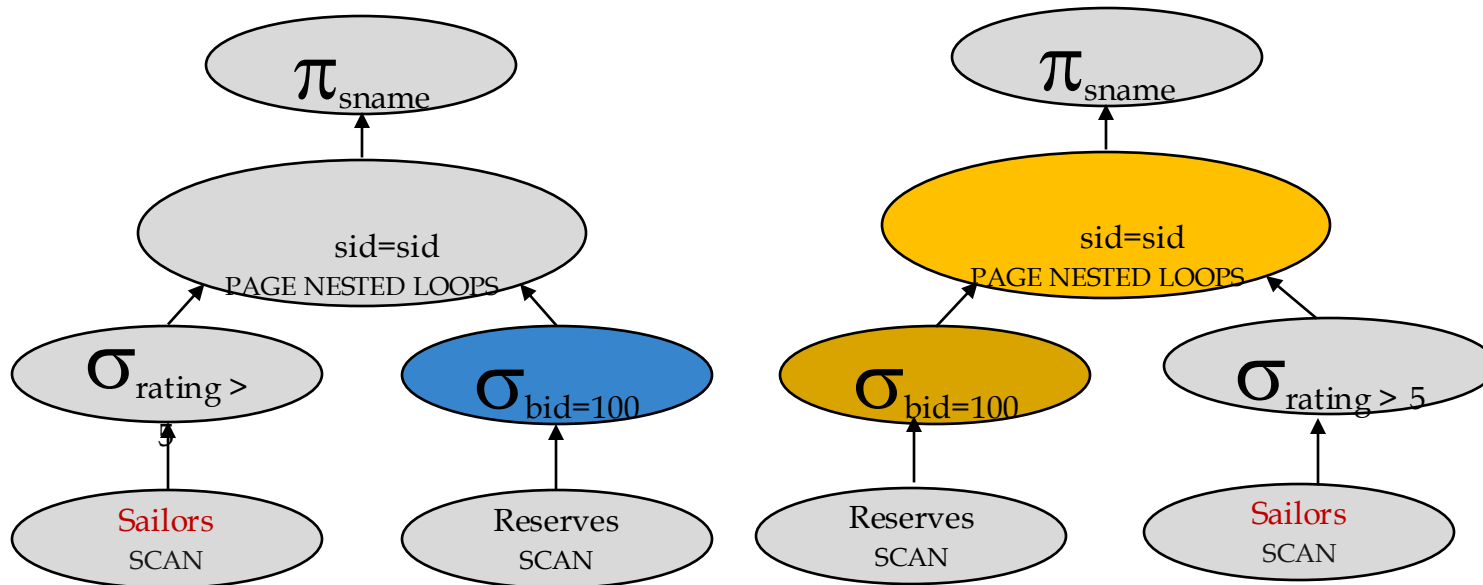


Join

- The join operations have a relatively high cost and are a key focus for query optimization. Here, we mainly discuss binary joins.
- 嵌套循环法 (nested loop)
- 利用索引或散列寻找匹配元组法(indexed)
- 排序归并(sort-merge)
- 散列连接法(hash-join)

Join1: Nested loop

Plan 3 Join Ordering



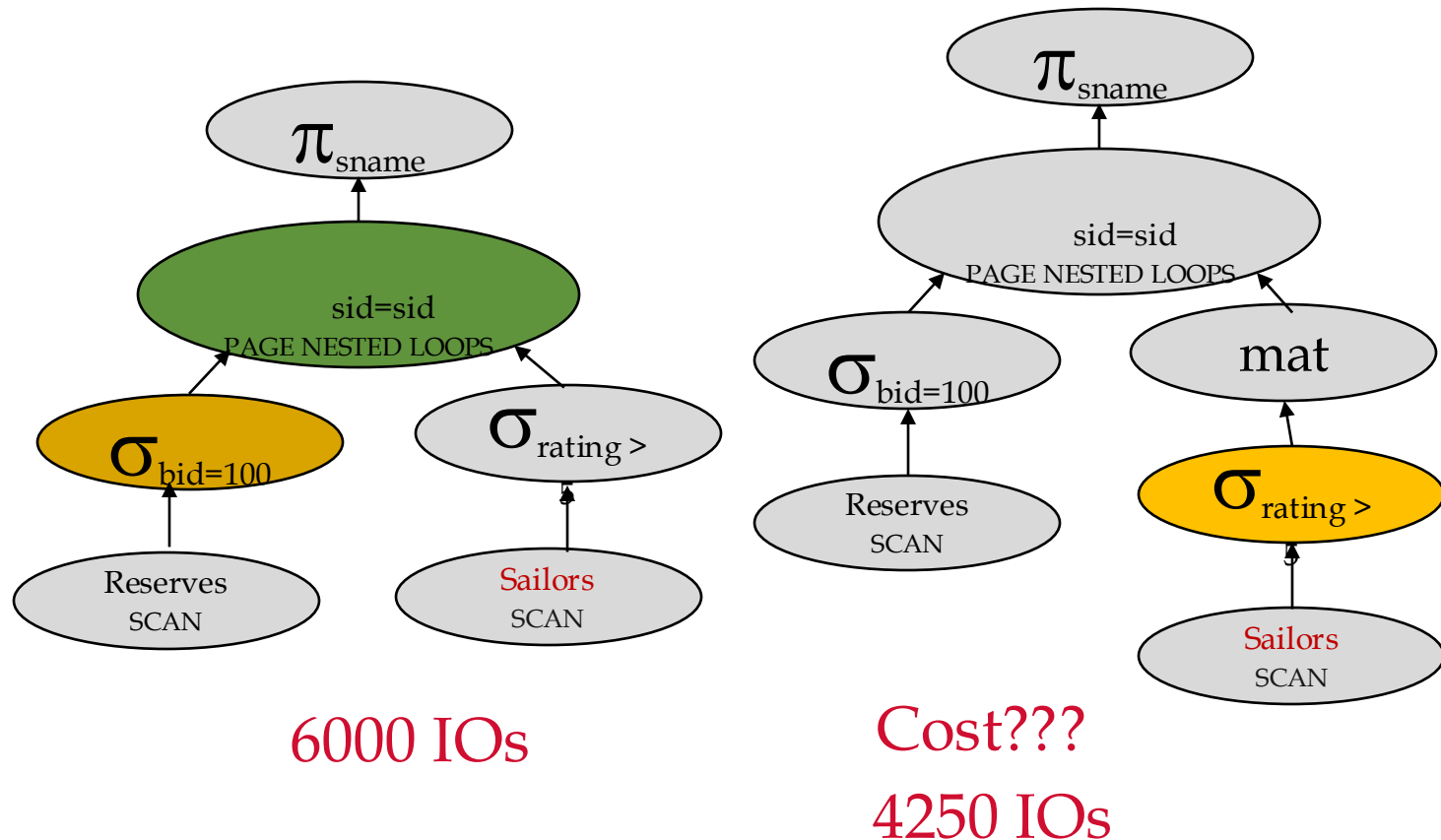
- Let's estimate the cost:
- Scan Reserves (1000 IOs)
- For each pageful of Reserves for bid 100, Scan Sailors (500 IOs)
- Total: $1000 + ??? * 500$
- Total: **$1000 + 10 * 500$**

250,500 IOs
 $500 + 250 * 1000$

6000 IOs



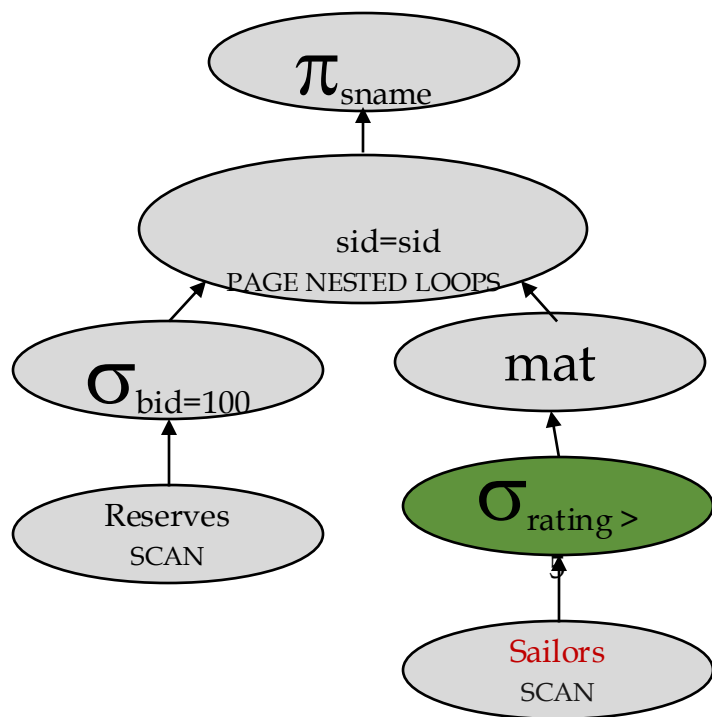
Plan4: Materializing Inner Loops



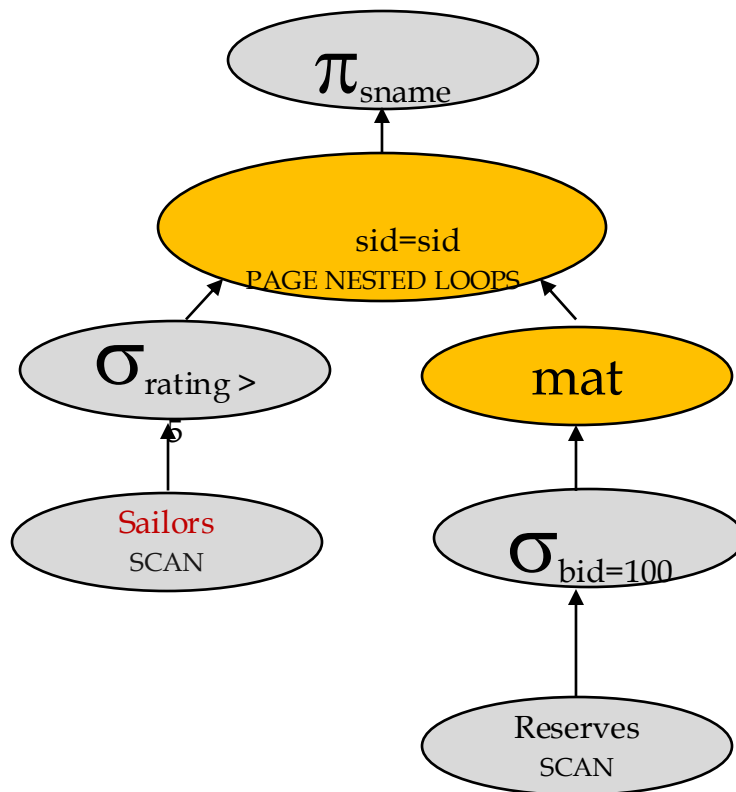
- Let's estimate the cost:
- Scan Reserves (1000 IOs)
- Scan Sailors (500 IOs)
- Materialize Temp table T1 (??? IOs)
- For each pageful of Reserves for bid 100, Scan T1 (??? IOs)
- Total: 1000 + 500 + ??? + 10*???
- 1000 + 500 + 250 + (10 * 250)



Plan5: Join Ordering Again



4250 IOs

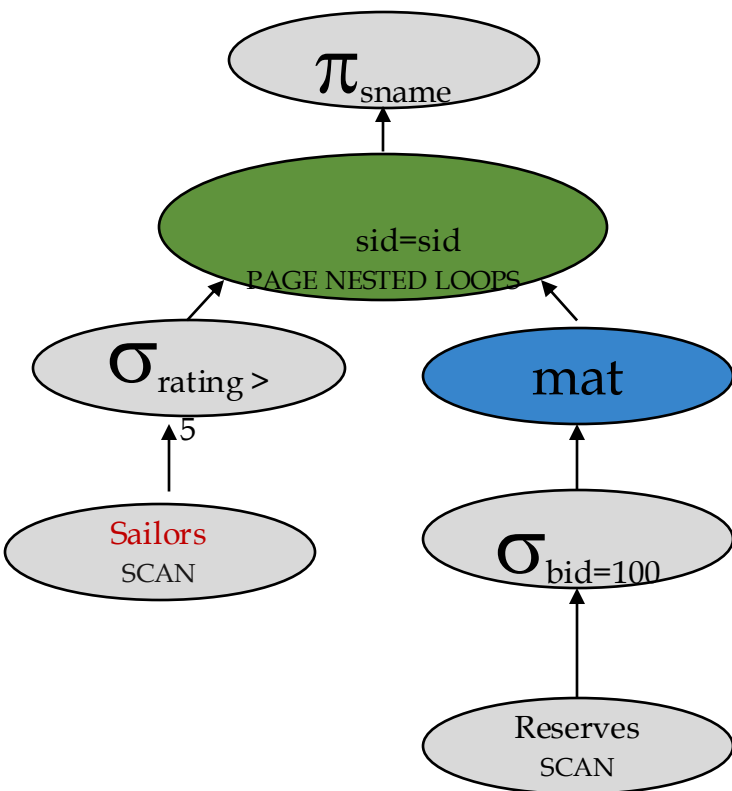


Cost???
4010 IOs

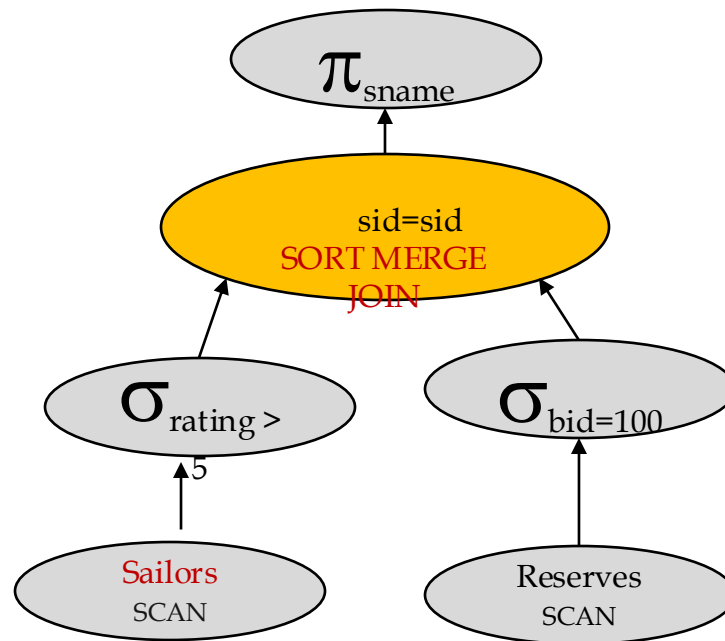
- Let's estimate the cost:
- Scan Sailors (500 IOs)
- Scan Reserves (1000 IOs)
- Materialize Temp table T1 (??? IOs)
- For each pageful of high-rated Sailors,
 Scan T1 (??? IOs)
- Total: 500 + 1000 + ??? + 250*???
- **500 + 1000 + 10 + (250 * 10)**



Plan6: Join Algorithm



4010 IOs



Cost???

- With 5 buffers, cost of plan
- Scan Reserves (1000)
- Scan Sailors (500)

- Sort high-rated sailors (???)

$$250 + 3 * 2 * 250$$

- Sort reservations for boat 100 (???)

$$10 + 2 * 10$$

- Merge (10+250) = 260

- Total:

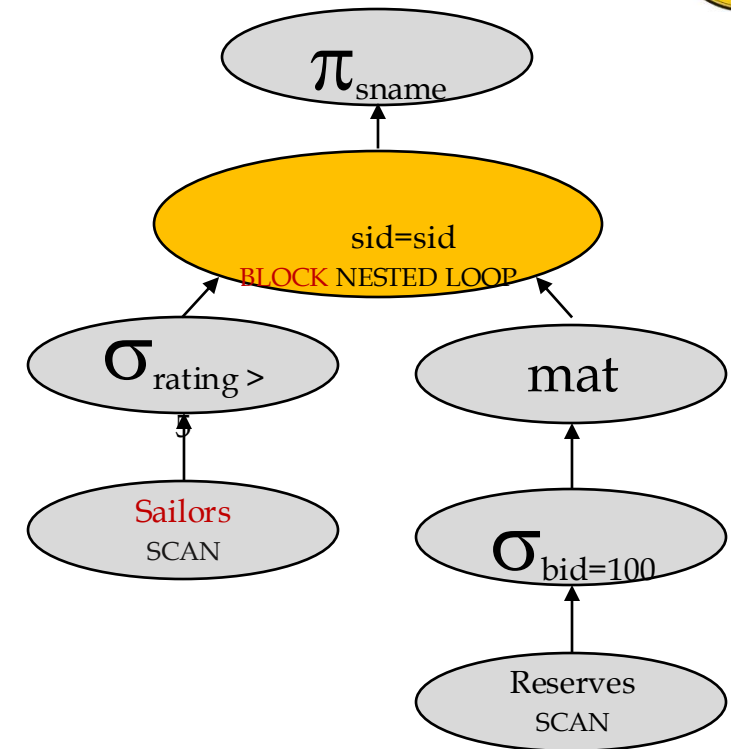
$$1000 + 500 + \text{sort reserves}(10 + 2 * 10) + \text{sort sailors}(250 + 3 * 2 * 250) + \text{merge}(10 + 250) = 3540$$



Plan7: Join Algorithm Again, Again.

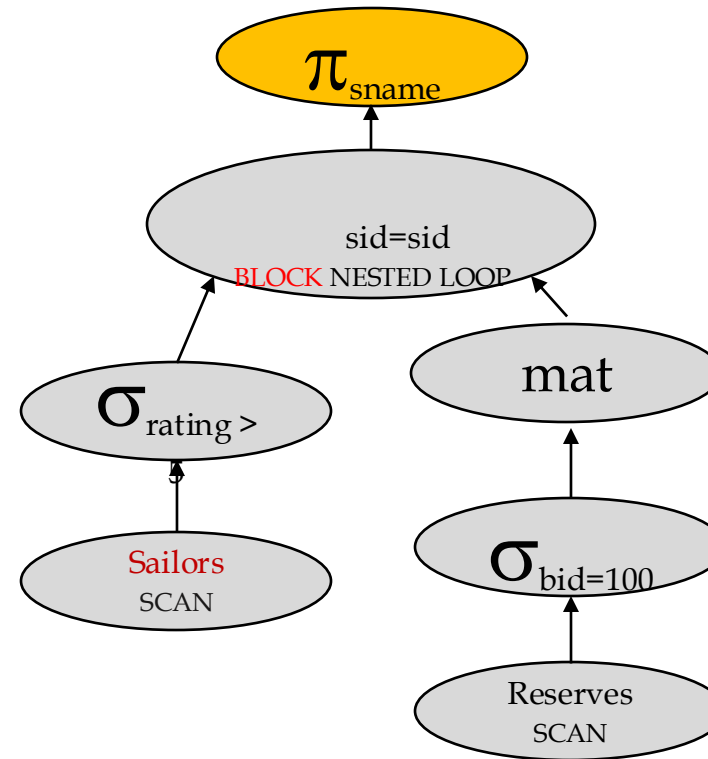
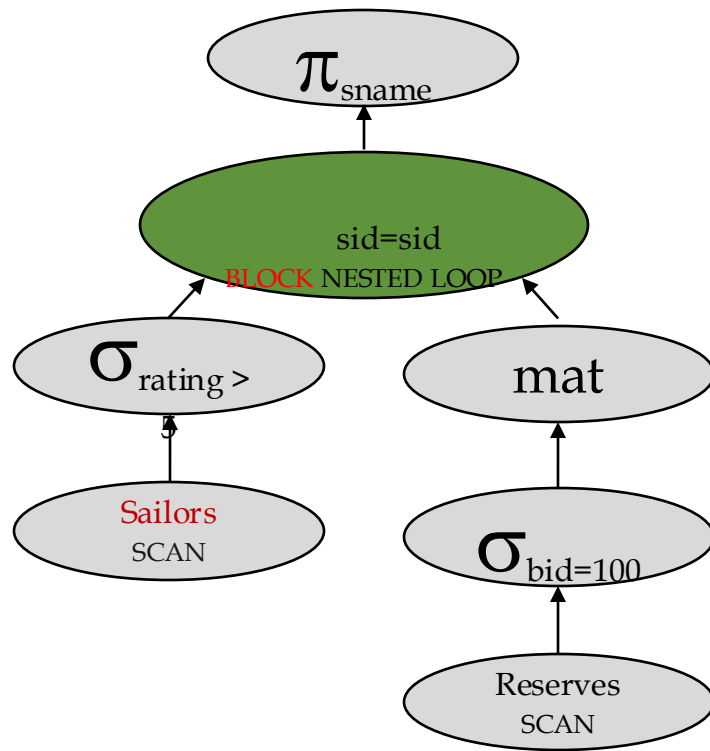
- With 5 buffers, cost of plan:
- Scan Sailors (500)
- Scan Reserves (1000)
- Write Temp T1 (10)
- For each **blockful** of high-rated sailors
- Loop on T1 (??? * 10)
- Total:

$$500 + 1000 + 10 + (\text{ceil}(250/3) * 10) = 500 + 1000 + 10 + (84 * 10) = 2350 \text{ IOs}$$





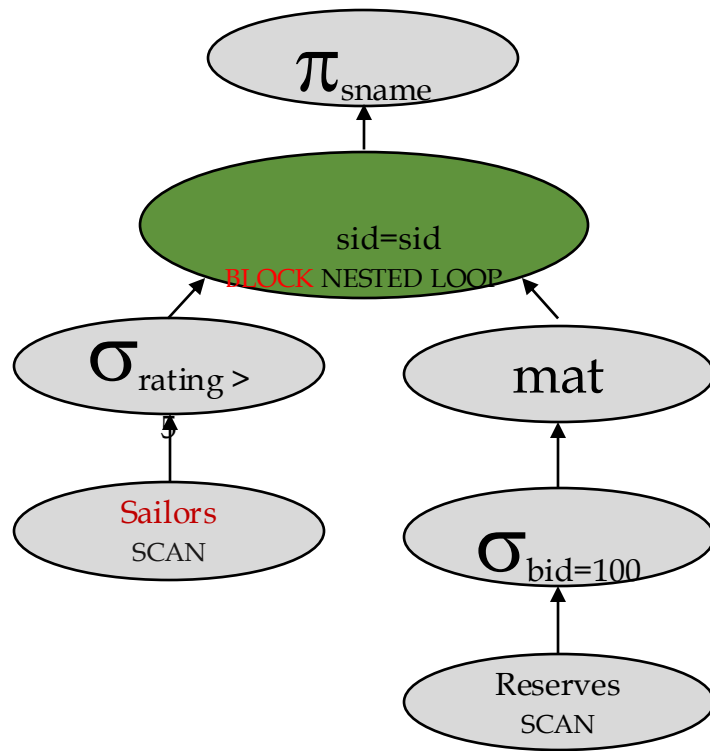
(look back): Projection Cascade & Pushdown



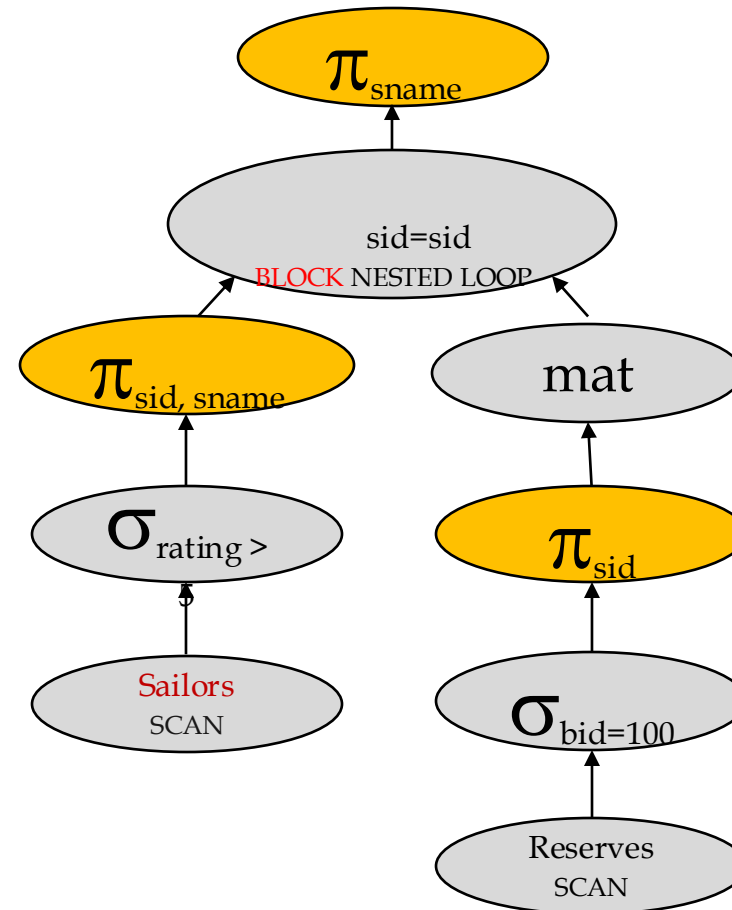
2350 IOs



(look back): Projection Cascade & Pushdown, cont



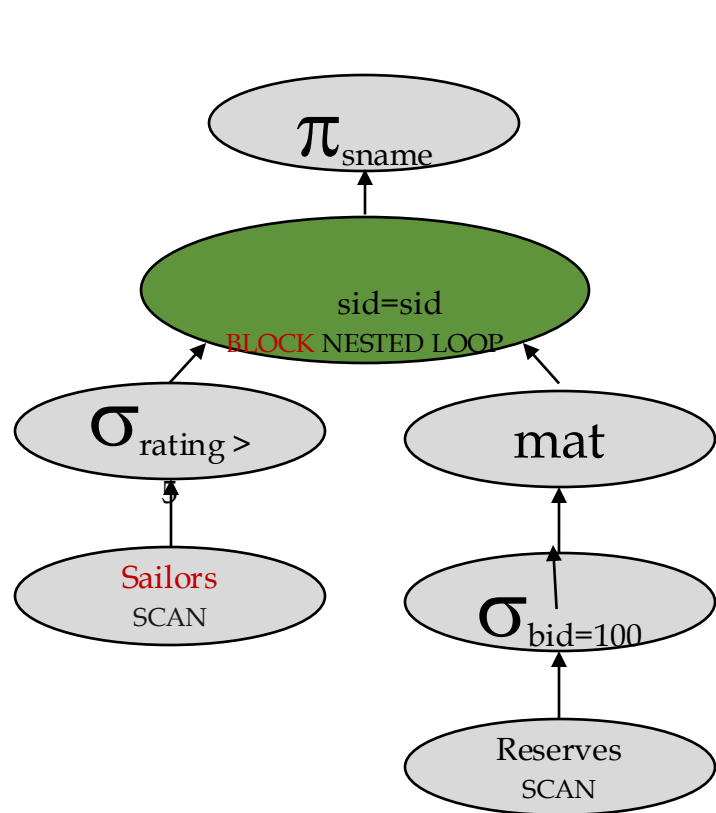
2350 IOs



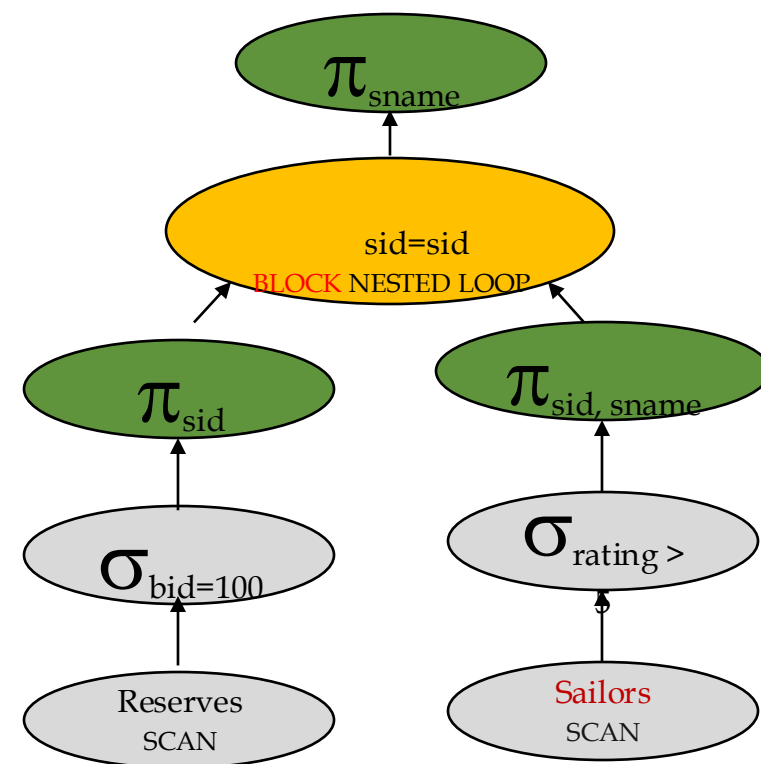
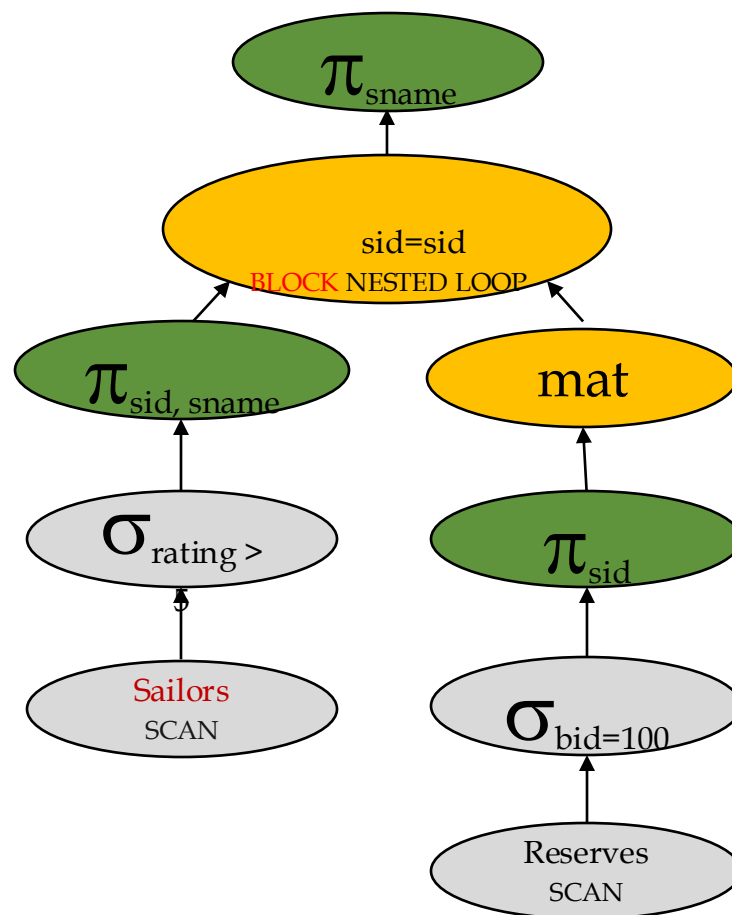




Plan 8: With Join Reordering, no Mat



2350 IOs



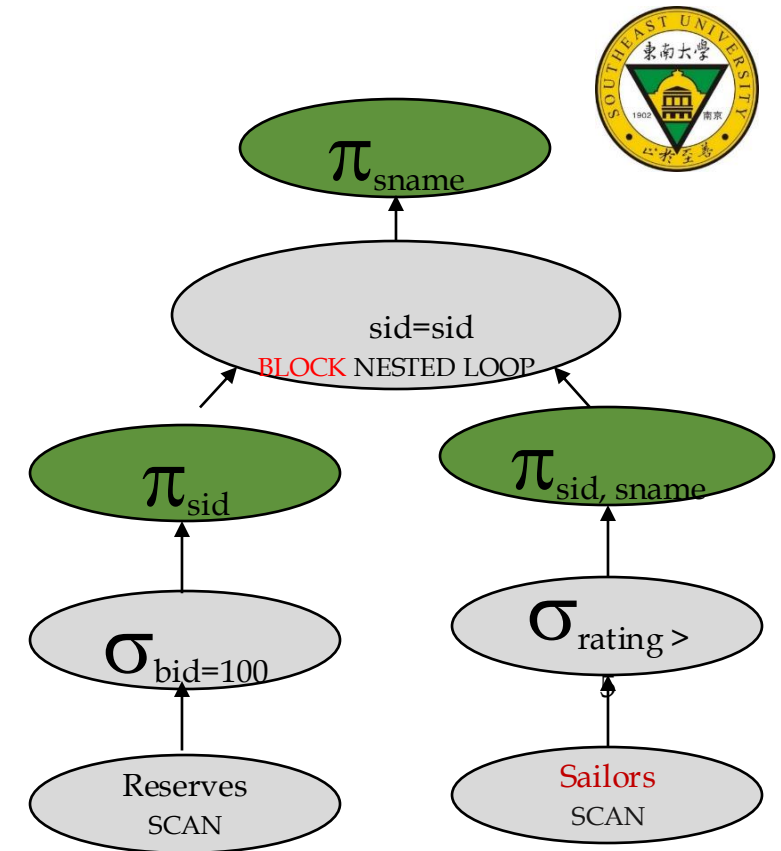


Plan 8: Cost Analysis

- With 5 buffers, cost of plan:
- Scan Reserves (1000)
- For each blockful of sids that rented boat 100
- (recall Reserve tuple is 40 bytes, assume sid is 4 bytes)
- Loop on Sailors (??? * 500)

$$(1000/100)/10 = 1!$$

- Total: 1500





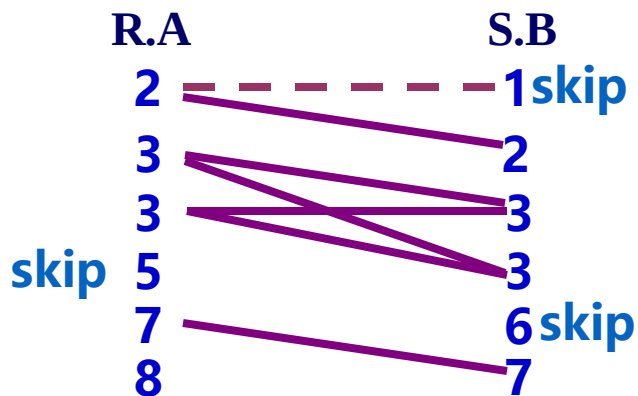
Join2: Using indexing or hashing to find matching tuples

- In the nested loop method, the inner relation undergoes multiple sequential scans. If there is *an appropriate access path* on the inner relation (such as an index or hash on the join attribute), sequential scans on the inner relation can be avoided, thereby reducing the number of I/O operations.
- *Question: If an index is built on the join attribute of the inner relation, will it necessarily improve the matching efficiency between the inner and outer relations?*
 - *When the number of matching tuples selected in each iteration constitutes a large proportion of the inner relation (for example, over 15%), using an unordered index may even be less efficient than a sequential scan.*
 - *When there is a clustered index on the join attribute of the inner relation, the index has the most significant impact on reducing the I/O operations required for the join.*



Join3: Sort-Merge

- Sort-merge join is a method used to combine rows from two sorted relations (tables)



```
i ← 1, j ← 1;
While(i ≤ n) and (j ≤ m)
do { if R(i)[A] > S(j)[B]
    then j ← j + 1
    else if R(i)[A] < S(j)[B]
    then i ← i + 1
    else {
        /* R(i)[A] = S(j)[B], 输出连接元组 */
        输出 < R(i), S(j) > 至 T;
        /* 输出 R(i) 和 S 中除 S(j) 外的其他元组所组成的连接元组 */
        l ← j + 1;
        While(l ≤ m) and (R(i)[A] = S(l)[B])
        do { 输出 < R(i), S(l) > 至 T;
            l ← l + 1; }
        /* 输出 S(j) 和 R 中除 R(i) 外的其他元组所组成的连接元组 */
        k ← i + 1;
        While(k ≤ n) and (R(k)[A] = S(j)[B])
        do { 输出 < R(k), S(j) > 至 T;
            k ← k + 1; }
        i ← i + 1, j ← j + 1; }
}
```



Join4: Hash join

- **Hash join:** because the join attributes of R and S have the same domain, R and S can be hashed into the same hash file using the same hash function, then $R \bowtie S$ can be computed based on the hash file.

- 关键在于建立一个供连接用的散列文件。

建立散列文件需要对R、S各扫描一次，且关系R和S一般不会对连接属性进行簇集。故而，每向散列文件加入一个元组，都需要一次I/O操作。

- 如何减少I/O次数？

可以在桶（散列文件）中不填入R、S的实际元组，而是代之以元组的tid，从而大大的缩小散列文件，使其有可能在内存中建立，而仅需对R、S各扫描一次。



Join4: Hash join

- （注意：同一桶中的元组未必都满足连接条件）。只需把桶中的匹配元组取出即可获得连接结果。
- 在取实际元组时，为减少物理块访问，可将各桶中，匹配元组的 tid 按块分类，一次集中取出同一块中所需的所有元组，当然这需要较大的内存开销。

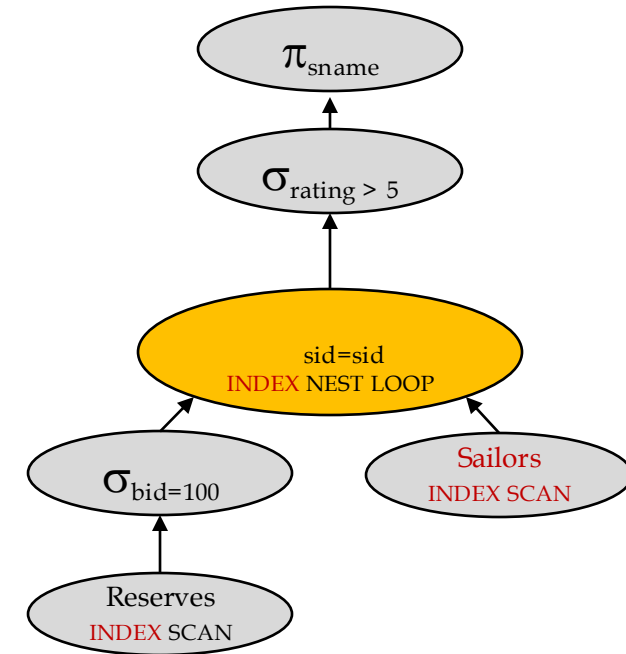
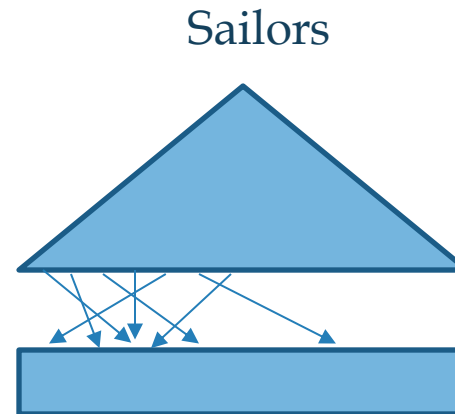
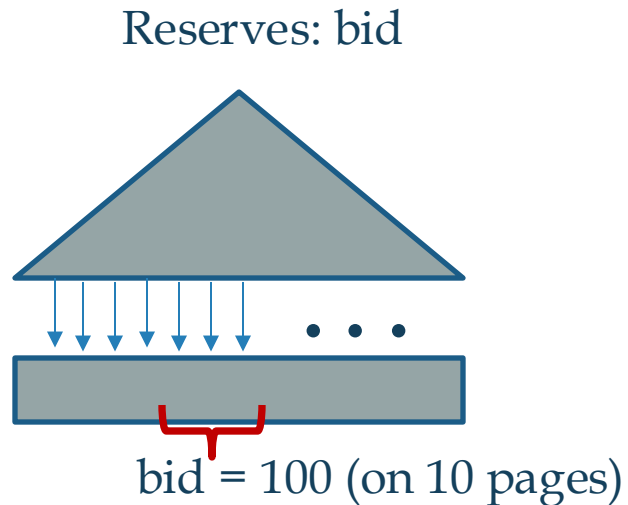


Some Common Heuristics: join

- 1) 两个关系都已按连接属性排序，则优先用排序归并法；两个关系中已有一个关系按连接属性排序，另一个关系较小，也可先对未排序关系按连接属性排序，再用排序归并法。
- 2) 两个关系中有一个关系在连接属性上有索引（特别是簇集索引）或散列，可以另一关系为外关系，顺序扫描，并利用内关系上的索引或散列寻找其匹配元组，以代替多遍扫描。
- 3) 不具备上述条件且关系较小，可用嵌套循环法。
- 4) 不具备1, 2, 3规则，可用散列连接法。

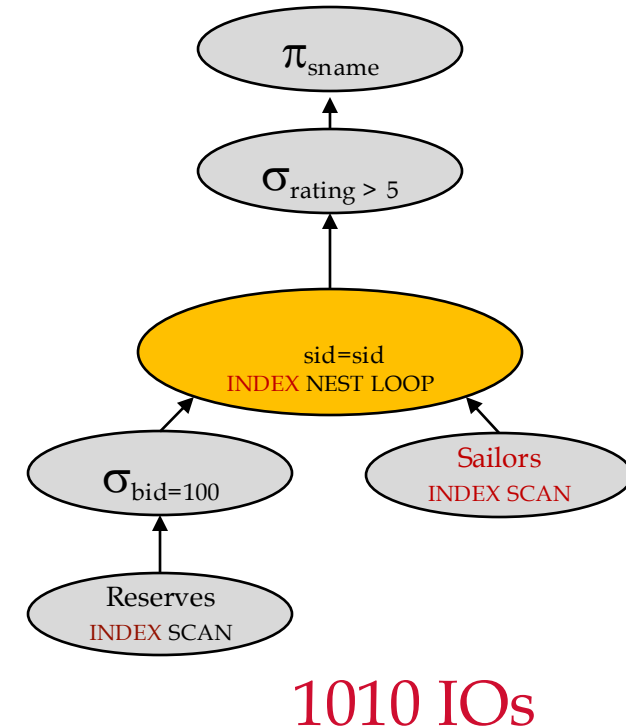
Examples: How About Indexes?

- Indexes:
 - Reserves.bid clustered
 - Sailors.sid unclustered
- Assume indexes fit in memory



Index Cost Analysis Part 2

- With clustered index on bid of Reserves, we access how many pages of Reserves?:
 - $100,000/100 = 1000$ tuples on $1000/100 = 10$ pages.
- for each Reserves tuple 1000
get matching Sailors tuple (1 IO)
(recall: 100 Reserves per page, 1000 pages)
- $10 + 1000 * 1$
- Cost: Selection of Reserves tuples (10 I/Os); then, for each, must get matching Sailors tuple (1000); total 1010 I/Os.





Projection

- Typically performed alongside selection and join operations, requiring no additional I/O overhead.
- If the set of projection attributes does not include the primary key, duplicate tuples may appear in the projection result.
- Duplicate tuples can be eliminated using methods such as sorting or hashing.
 - Hash the projection results into a file based on one or more attributes. When a tuple is hashed into a bucket, it can be checked for duplication with tuples already in the bucket.

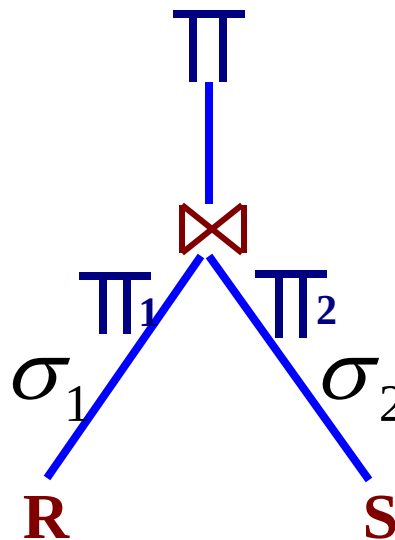


Set Operations

- Common set operations: Cartesian product, union, intersection, difference, etc.
- Avoid Cartesian product.
- If relations R and S are compatible, to perform union (intersection, difference) operations on R and S, you can first sort R and S by the same attribute (typically the primary key), and then scan both relations to select the required tuples.
- Hashing is another method for solving the union, intersection, and difference operations mentioned above.

Combined Operations

- Sometimes, multiple operations are combined and performed simultaneously, such as *combining projection and selection operations (with duplicate tuples being eliminated separately)* to improve efficiency. It is also possible to combine and execute multiple operations on a larger scale.





Summing up

- Optimizing before execution is called *static optimization*, which can only utilize statistical data and may sometimes not be accurate.
- Optimizing during query execution is called *dynamic optimization*. It *estimates costs* based on actual execution results, making it more aligned with reality. However, it requires optimization every time a query is executed, which is not suitable for compilation-based implementations and increases execution time. It can only utilize statistical data, which may sometimes be inaccurate. Moreover, during optimization, waiting for intermediate results increases waiting time and data dependency, which is detrimental to parallelism.



Cost Estimation*

- For each plan considered, must estimate total cost:
 - Must estimate **cost** of each operation in plan tree.
 - Depends on input cardinalities.
 - We've already discussed this for various operators
 - sequential scan, index scan, joins, etc.
 - Must estimate **size of result** for each operation in tree!
 - Because it determines downstream input cardinalities!
 - Use information about the input relations.
 - For selections and joins, assume independence of predicates.



Statistics and Catalogs

- Need info on relations and indexes involved.

- **Catalogs** typically contain at least:

Statistic	Meaning
NTuples	# of tuples in a table (cardinality)
NPages	# of disk pages in a table
Low/High	min/max value in a column
Nkeys	# of distinct values in a column
IHeight	the height of an index
INPages	# of disk pages in an index

- Catalogs updated periodically.
 - Too expensive to do continuously
 - Lots of approximation anyway, so a little slop here is ok.
- Modern systems do more
 - Esp. keep more detailed statistical information on data values
 - e.g., *histograms*



Cost Estimates for Single-Relation Plans

- Index I on primary key matches selection:
 - *Cost is $\text{Height}(I)+1$ for a B+ tree, about 1.2 for hash index.*
- Clustered index I matching one or more selects:
 - *$(\text{NPages}(I)+\text{NPages}(R)) * \text{product of RF's (Reduction Factor) of matching selects.}$*
- Non-clustered index I matching one or more selects:
 - *$(\text{NPages}(I)+\text{NTuples}(R)) * \text{product of RF's of matching selects.}$*
- Sequential scan of file:
 - *$\text{NPages}(R)$.*
- + **Note:** *Typically, no duplicate elimination on projections! (Exception: Done on answers if user says DISTINCT.)*

Queries Over Multiple Relations

- Fundamental decision in System R: *only left-deep join trees* are considered.
- As the number of joins increases, the number of alternative plans grows rapidly; ***we need to restrict the search space***. Left-deep trees allow us to generate all *fully pipelined* plans.
- Intermediate results not written to temporary files.
Not all left-deep trees are fully pipelined (e.g., SortMerge join).

